

DELTA TRACE: EFFICIENT SIGNED ATTRIBUTION FOR REASONING LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Reasoning models combine evidence from many parts of an input. A source can shape a response through the information it supplies and through its role in selecting or retaining other information. We present DELTA TRACE, a method that traces these roles back to the input as signed contributions. For a fixed response, the method follows the score change between an original input and a reference. In attention, it separates changes in the content being carried from changes in the weights that select that content. In gated memory, it follows what is retained, read, and written. The contributions preserve their sign as they pass through the model and sum to the response-score change. The computation accumulates attention contributions in tiles and memory contributions in chunks, giving sequence-linear auxiliary storage at fixed model dimensions. This construction applies to attention and hybrid reasoning models. The evaluation connects source recovery and response faithfulness with complete attribution cost.

1 INTRODUCTION

Reasoning language models use input evidence across an extended sequence of computations. An explanation connects their answers to the sources that shape those answers. For a given response, a useful attribution identifies both the amount of contribution assigned to a source and its direction. Supporting and opposing evidence can coexist in the same context, and their contributions can cancel in the final prediction. Preserving this structure makes the explanation informative about how the model combines evidence.

Evidence shapes a model’s answer through what it says and how the model uses it. A passage may supply a fact, while a word elsewhere in the context changes which fact is selected. Attention carries selected information to later positions; memory can retain it for later use. FlashTrace traces source importance for multi-token targets through span aggregation and recursive attribution (Pan et al., 2026). DELTA TRACE connects these roles to a shared response score, assigning signed contributions to the sources that supply content and guide its selection, retention, and use.

DELTA TRACE explains a response by following the change from a reference input to the original input. Each model operation distributes its output change among the changes of its inputs. A reverse traversal composes these local distributions into a contribution for every input source. The same construction handles a small activation change and a transition across a nonlinear region: propagation uses the change across the two executions. This gives the method a direct connection to the response being explained and a common scale for supporting and opposing contributions.

The key design follows the model’s use of evidence. In attention, a change in a source’s content travels with the weight assigned to that source in the original execution. Changes in the selection itself receive their own contribution. Normalization connects this selection to competition among the sources in an attention row. In gated memory, the same construction follows what is kept, read, and written. Shared rules carry these contributions through the remaining operations, preserving their sum as they reach the input.

The method follows the model’s efficient computations. Attention contributions accumulate in FlashAttention tiles (Dao et al., 2022); memory contributions use chunk states and batched products. Each tile or chunk adds to the source coefficients, keeping auxiliary storage linear in sequence length at fixed model dimensions.

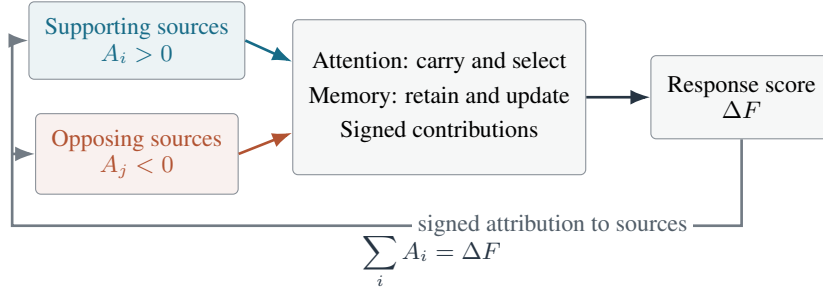


Figure 1: **From a response to its evidence.** Attention selects and carries information; memory retains and updates it. DELTATRACE follows these operations backward to assign signed contributions to input sources. Supporting and opposing contributions share a common response target and sum to its score change.

This work develops three connected contributions. First, DELTATRACE traces signed response contributions through content and control with fixed rules shared across model architectures (Section 2). Second, analytic attention and memory rules connect this propagation to a finite chain rule and contribution conservation (Sections 2.3–2.4). Third, a tiled and chunked computation realizes these rules with sequence-linear auxiliary storage at fixed model dimensions (Section 3). The evaluation protocol links source recovery and faithfulness to complete attribution cost (Section 4).

2 DELTATRACE

2.1 WHAT CONTRIBUTES TO A RESPONSE

A question about an event’s date may draw on a date in one sentence and a qualifier in another. The date supplies content; the qualifier can change which date the model selects. Both roles can shape the answer. DELTATRACE traces these contributions through the model, connecting the response to the input words that supply evidence and guide its use (Figure 1).

The explanation concerns a fixed response y . Its score is the sum of the response tokens’ log-probabilities,

$$F(x; y) = \sum_{t \in \mathcal{T}} \log p_\theta(y_t \mid x, y_{<t}), \quad (1)$$

where $\mathcal{T}$ selects the response positions. DELTATRACE compares the original input x_1 with a reference x_0 , keeping the response fixed. This comparison defines the change being explained: $\Delta F = F(x_1; y) - F(x_0; y)$. In our evaluation, the reference replaces eligible source tokens with EOS. Each source receives a signed share of this change. A positive share contributes toward the increase in the response score; a negative share contributes in the opposite direction.

2.2 FOLLOWING EVIDENCE BACKWARD

DELTATRACE first runs the model on the original and reference inputs, recording how its internal representations change. It then follows the response backward through the computation. At each operation, it distributes the contribution arriving at the output among the inputs that produced that output. Attention sends contributions to the positions it reads; memory carries them through earlier writes; residual connections divide them among branches. Contributions that reach the same input are added together.

Each local rule accounts for the change across the two executions. For a nonlinear activation, this means relating its observed output change to its input change across that interval. For a weighted sum, the rule follows the weights and the changes in the vectors being combined. Composing these rules gives the coefficient m_i of each source’s embedding change Δe_i , yielding

$$A_i = \langle m_i, \Delta e_i \rangle. \quad (2)$$

Thus a source receives credit through every computational path connecting it to the response. The sign is carried through those paths, so supporting and opposing contributions can meet and cancel. The local accounting preserves the total response change:

$$\sum_i A_i = F(x_1; y) - F(x_0; y). \quad (3)$$

This places the source scores on a common scale. Appendix A.1 gives the finite chain rule and conservation theorem.

2.3 WHAT ATTENTION CARRIES AND SELECTS

An attention head combines the content at source positions using weights chosen by queries and keys. A source can therefore affect the output by changing what is carried, how strongly it is selected, or both. In the date example, the value representation carries information about the date, while the attention weights determine how much of that information reaches a later position. DELTATRACE follows both routes to the response.

For attention probabilities P and values V , the output is $Y = PV$. Its change separates into

$$\Delta Y = \underbrace{P_1 \Delta V}_{\text{content change}} + \underbrace{\Delta P V_0}_{\text{routing change}}. \quad (4)$$

The content term uses the attention chosen for the original input. A change in a source’s value therefore travels with the weight that the model assigns to it in that execution. The routing term measures the effect of redistributing attention over the reference values. When content and selection change together, their interaction is assigned to the content term. These two terms give a complete account of the attention output change while retaining the contribution of selection itself.

Attention also couples the sources within each row: increasing the share assigned to one source redistributes weight among the others. DELTATRACE carries this competition backward through normalization. If u_i is the coefficient arriving at an attention probability, its score coefficient is

$$m_{z,i} = \ell_i(u_i - \bar{u}_\ell), \quad \bar{u}_\ell = \frac{\sum_j \ell_j u_j}{\sum_j \ell_j}. \quad (5)$$

The positive weight ℓ_i is the logarithmic mean of that source’s probabilities in the two executions. Subtracting the weighted mean expresses each source’s contribution relative to the alternatives in the same row. Query and key coefficients then carry this routing contribution to the representations that selected the sources. The same normalization construction connects the response’s log-probability to its output logits. Appendix A.2 specifies the logarithmic mean and derives the finite identity.

2.4 WHAT MEMORY KEEPS AND UPDATES

In a hybrid model, evidence can also reach a response through a memory state. Information written at an earlier position can survive several updates and be read by a later query. For the date example, the relevant content may be written when the model processes the source sentence and read again at a later position. The explanation therefore follows both the stored content and the gates that determine its survival and use.

Gated delta memory combines retained information with a new write (Yang et al., 2025):

$$S_t = \underbrace{\alpha_t S_{t-1}}_{\text{retained evidence}} + \underbrace{k_t u_t^\top}_{\text{new write}}. \quad (6)$$

The retention gate α_t controls how much of the previous state remains. The write vector u_t updates the content associated with key k_t : it is the difference between the incoming value and the value read from retained memory, scaled by a write gate. A query reads the resulting state to produce the token’s memory output.

DELTATRACE reverses this sequence of reading, writing, and retention. At a read, contributions pass to the stored content and to the query that retrieves it. At a write, they pass to the incoming value, the retrieval key, and the write gate. Along the retained state, they continue to earlier memory

contents and to the retention gate. Each step uses the same content-and-control allocation as attention: changing content is weighted by the original execution’s control, and control changes receive their own contribution. This lets an earlier source receive a signed contribution through the memory operations that connect it to the response. Appendix A.4 gives the complete recurrence.

Shared rules across models. The same operator uses the same propagation rule in Qwen3 and Qwen3.5. Attention values use Equation 4; query–key products and SwiGLU divide their interaction symmetrically; memory and output gates use the content-and-control allocation described above. Linear layers, residual additions, and normalization complete the traversal through each architecture. Appendix A.3 collects the fixed rules and their formulas.

3 EFFICIENT COMPUTATION

Accumulating attention contributions in tiles. The attention mechanism produces a contribution for each source vector and routing score. DELTATRACE accumulates these contributions a tile at a time using the FlashAttention framework (Dao et al., 2022). Within each tile, it reconstructs the attention probabilities, computes the required weighted sums, and adds the result to the source coefficients. The tile can then be reused. The quantities carried across tiles are vectors and row statistics, which grow linearly with the number of tokens.

The weighted mean in Equation 5 supplies the information shared across an attention row. A first sweep accumulates this mean and its normalizer. Subsequent sweeps use those statistics to propagate contributions to queries, keys, and values. This organization puts the content and routing computations into a fixed number of tiled sweeps. For sequence length T and head dimension d , auxiliary attention storage is $O(Td)$ and dense attention arithmetic is $O(T^2d)$ per head.

Following memory through chunks. Memory passes information between consecutive chunks through a boundary state. DELTATRACE follows the contribution to that state backward, then distributes it among the reads, writes, and gates inside each chunk. The implementation uses FLA’s native chunk-state adjoints and batched matrix products to perform these operations. The retention contribution is accumulated with an associative scan. At fixed state dimensions and chunk size, the stored token contributions and boundary states grow linearly with sequence length. Appendix B gives the tensor accounting and execution details.

Complete attribution. The full procedure consists of the two model executions, layer replay, backward contribution propagation, and transfers. Layer replay supplies the activations needed by the current layer, while the tiled attention and chunked memory computations accumulate its input contributions. Together, these steps apply the same attribution method to attention and hybrid models. The evaluation measures their combined latency, memory use, and throughput.

4 EVALUATION PROTOCOL

Source recovery and faithfulness. The evaluation uses the processed tasks and fixed generated trajectories from the authors’ `table1-data-v1` release accompanying FlashTrace (Pan et al., 2026).¹ The source-recovery protocol measures recall of gold evidence within the top 10% of eligible input tokens. The faithfulness protocol uses the authors’ 20-step deletion procedure and the released RISE and MAS scoring functions. Both metrics use the positive part of DELTATRACE’s source contributions, and the complete signed attribution is retained as a separate output. Recovery and faithfulness are reported alongside the signed evidence visualizations.

Aligned comparisons. The Qwen3-8B comparison uses the original input formatting, FP16 model loading, response targets, EOS handling, and eager scoring defined by the released experiment code. The authors’ `table1-data-v1` release supplies the reference results for complete released task selections, using one recursive hop for faithfulness and three hops for recovery. The Qwen3.5-9B

¹<https://github.com/wbopan/flashtrace/releases/tag/table1-data-v1>

comparison uses the same data and scoring construction and runs the fixed FlashTrace implementation on the same Qwen3.5 model. Both DELTATRACE profiles use the shared rules described in Section 2, with the complete operator specification in Appendix A.3.

Cost measurements. The measurement protocol records complete attribution latency, peak allocated memory, and throughput for real sample batches. Sequence length, model, dtype, and batch composition accompany each measurement. Loading, compilation, warmup, and measured execution are recorded separately. The cost breakdown follows the stages in Section 3, connecting the observed resource use to the propagation algorithm.

Mechanism comparisons. The central attention comparison fixes all other operators and evaluates the content-oriented identity in Equation 4 alongside symmetric and reverse-order product allocation. The same official examples and metrics connect the choice of interaction assignment to evidence recovery and response faithfulness. Signed source visualizations show how supporting and opposing contributions combine for the response target.

5 RELATED WORK

Prediction attribution. Integrated Gradients studies attribution through sensitivity and implementation invariance (Sundararajan et al., 2017). DeepLIFT propagates differences from reference activations and develops multiplier composition with a summation-to-delta property (Shrikumar et al., 2017). DELTATRACE develops finite propagation for normalized attention and gated memory, with explicit content and control allocations and a tiled implementation. These operator-level constructions connect its prediction target to the structure of contemporary reasoning models.

Reasoning and efficient sequence computation. FlashTrace combines span aggregation and recursive attribution to trace importance through reasoning sequences (Pan et al., 2026). Its released tasks, trajectories, and evaluation code provide the comparison protocol used here. FlashAttention organizes exact attention around tiled computation and memory traffic (Dao et al., 2022). Gated Delta Networks combine memory decay with delta-rule updates (Yang et al., 2025). DELTATRACE brings signed contribution propagation to both forms of sequence computation through their respective tiled and recurrent structures.

6 CONCLUSION

DELTATRACE traces signed contributions from a reasoning model’s response to its input evidence. Its fixed propagation rules connect content, attention routing, and gated memory to a common response change. Analytic attention normalization and finite state recurrence support contribution conservation, while tiled and chunked computation give the method sequence-linear auxiliary storage at fixed model dimensions. Together, these components establish a unified computational basis for signed evidence attribution in attention and hybrid language models.

AI USE STATEMENT

AI assistance supported conceptual development, mathematical derivations, research code, experimental design and analysis, literature retrieval, manuscript organization, and language editing. The manuscript uses the released evaluation data and model-generated trajectories specified in the evaluation protocol.

REFERENCES

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Wenbo Pan, Zhichao Liu, Xianlong Wang, Haining Yu, and Xiaohua Jia. Towards long-horizon interpretability: Efficient and faithful multi-token attribution for reasoning LLMs. *arXiv preprint arXiv:2602.01914*, 2026. URL <https://arxiv.org/abs/2602.01914>.
- Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3145–3153. PMLR, 2017. URL <https://proceedings.mlr.press/v70/shrikumar17a.html>.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3319–3328. PMLR, 2017. URL <https://proceedings.mlr.press/v70/sundararajan17a.html>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2412.06464>.

A FORMAL RULES AND DERIVATIONS

A.1 FINITE PROPAGATION AND CONTRIBUTION CONSERVATION

For an operation $v = f(u_1, \dots, u_k)$, DELTATRACE constructs linear maps D_j from the two executions such that

$$\Delta v = \sum_{j=1}^k D_j \Delta u_j. \quad (7)$$

An upstream coefficient m_v propagates to input j as $D_j^\top m_v$. Coefficients from multiple consumers add at their shared input. Starting with coefficient 1 at the scalar target, the reverse traversal produces the source coefficients in Equation 2. For a scalar nonlinearity f , the local coefficient is the divided difference $(f(u_1) - f(u_0))/(u_1 - u_0)$, with its derivative as the coincident-input limit. Linear transformations retain their weights, and addition sends the coefficient to each summand.

Theorem 1 (Contribution conservation). *For an acyclic computation over real-valued operations satisfying Equation 7, reverse propagation from a scalar target yields the conservation identity in Equation 3.*

Proof. An acyclic computation can be expanded into elementary operations with explicit branches. At an operation $v = f(u_1, \dots, u_k)$, Equation 7 gives

$$\langle m_v, \Delta v \rangle = \sum_j \langle D_j^\top m_v, \Delta u_j \rangle. \quad (8)$$

Replacing a node’s contribution by the contributions of its parents therefore preserves the total on a cut through the graph. At shared parents, coefficients from different consumers sum. Repeated substitution reaches the input cut and yields Equation 3, since the coefficient of the scalar target is 1. Constant inputs have zero difference and contribute zero to this sum. $\square$

A.2 FINITE ATTENTION NORMALIZATION

For an attention row $p_b = \text{softmax}(z_b)$, $b \in \{0, 1\}$, the logarithmic mean and its sum are

$$\ell_i = \frac{p_{1,i} - p_{0,i}}{\log p_{1,i} - \log p_{0,i}}, \quad \tau = \sum_i \ell_i, \quad (9)$$

with $\ell_i = p_{0,i}$ at coincident probabilities. All quantities are evaluated on the common unmasked support.

Proposition 1 (Finite attention normalization). *For finite logits on a common support,*

$$\Delta p = \left(\text{diag}(\ell) - \frac{\ell \ell^\top}{\tau} \right) \Delta z. \quad (10)$$

Proof. The common support contains positive probabilities. For endpoint b , $\log p_b = z_b - a_b \mathbf{1}$, where $a_b = \log \sum_i \exp(z_{b,i})$. The definition of the logarithmic mean gives

$$\Delta p = \ell \odot (\Delta z - \Delta a \mathbf{1}). \quad (11)$$

Probability normalization implies $\mathbf{1}^\top \Delta p = 0$, so

$$\Delta a = \frac{\ell^\top \Delta z}{\tau}. \quad (12)$$

Substitution proves Equation 10. For a target coordinate y ,

$$\Delta \log p_y = \left(e_y - \frac{\ell}{\tau} \right)^\top \Delta z. \quad (13)$$

The coincident-probability definition follows from the continuous limit of the logarithmic mean. The score operator is symmetric positive semidefinite: for $w_i = \ell_i/\tau$,

$$u^\top \left(\text{diag}(\ell) - \frac{\ell\ell^\top}{\tau} \right) u = \tau \left(\sum_i w_i u_i^2 - \left(\sum_i w_i u_i \right)^2 \right) \geq 0. \quad (14)$$

This is the weighted variance of u multiplied by τ .

□

The operator is symmetric. Its transpose action gives Equation 5, and the response coefficient for a log-softmax target is $m_z = e_y - \ell/\tau$.

A.3 SHARED OPERATOR RULES

For attention output $Y = PV$, Equation 4 gives the coefficients $m_V = P_1^\top U$ and $m_P = UV_0^\top$ for an upstream coefficient U .

Query–key scores use the symmetric product identity,

$$\Delta(QK^\top) = \Delta Q \bar{K}^\top + \bar{Q} \Delta K^\top, \quad \bar{Q} = (Q_0 + Q_1)/2, \quad (15)$$

with $\bar{K}$ defined in the same way. The common operator rules appear in Table 1. They depend on the operator and its role, and are shared by the two model families. Normalization, rotary position transformations, and grouped heads follow their respective model definitions.

Table 1: Fixed propagation rules shared across architectures. Bars denote endpoint averages. Gated-memory operations extend the same content-oriented rule to recurrent states.

Operation	Change decomposition
Linear and residual	$\Delta(Wx + b) = W\Delta x; \Delta(a + b) = \Delta a + \Delta b$
Attention values	$\Delta(PV) = P_1\Delta V + \Delta PV_0$
Attention scores	$\Delta(QK^\top) = \Delta Q \bar{K}^\top + \bar{Q} \Delta K^\top$
SwiGLU	$\Delta(us) = \bar{s} \odot \Delta u + \bar{u} \odot \Delta s, s = \text{SiLU}(g)$
Output gating	$\Delta(ns) = s_1 \odot \Delta n + n_0 \odot \Delta s$
RMS normalization	Analytic finite rule for $wx/\sqrt{\text{mean}(x^2) + \epsilon}$

A.4 GATED-MEMORY RECURRENCE

With column vectors and a key-by-value state $S_t \in \mathbb{R}^{d_k \times d_v}$, its per-token computation is

$$\begin{aligned} T_t &= \alpha_t S_{t-1}, & r_t &= T_t^\top k_t, \\ c_t &= v_t - r_t, & u_t &= \beta_t c_t, \\ S_t &= T_t + k_t u_t^\top, & o_t &= S_t^\top q_t. \end{aligned} \quad (16)$$

The query includes the model’s fixed attention scale. The state T_t contains retained evidence, r_t reads existing content, and u_t writes the update. DELTATRACE follows the change of this computation using fixed content-oriented products:

$$\begin{aligned} \Delta T_t &= \alpha_{t,1} \Delta S_{t-1} + S_{t-1,0} \Delta \alpha_t, \\ \Delta r_t &= (\Delta T_t)^\top k_{t,1} + T_{t,0}^\top \Delta k_t, \\ \Delta u_t &= \beta_{t,1} (\Delta v_t - \Delta r_t) + c_{t,0} \Delta \beta_t, \\ \Delta S_t &= \Delta T_t + k_{t,1} \Delta u_t^\top + \Delta k_t u_{t,0}^\top, \\ \Delta o_t &= (\Delta S_t)^\top q_{t,1} + S_{t,0}^\top \Delta q_t. \end{aligned} \quad (17)$$

This recurrence assigns contributions to new values, retrieval keys, queries, and both gates. Reversing it carries the response coefficient through stored evidence to earlier sources. Scalar gate nonlinearities and query/key normalization use the same finite propagation construction as the surrounding network.

A.5 PRODUCT AND NORMALIZATION IDENTITIES

For a bilinear map B , both

$$\Delta B(a, b) = B(\Delta a, \bar{b}) + B(\bar{a}, \Delta b), \quad (18)$$

$$\Delta B(a, b) = B(\Delta a, b_1) + B(a_0, \Delta b) \quad (19)$$

follow by expansion. Equation 4, the shared SwiGLU rule, and every product in Equation 17 are direct instances with the stated argument ordering. Subtraction and state addition are linear, completing the finite memory recurrence.

For RMS normalization $f(x) = w \odot x/r(x)$, $r(x) = \sqrt{\|x\|^2/d + \epsilon}$, the symmetric product rule gives an analytic coefficient. With $\bar{x} = (x_0 + x_1)/2$, $r_b = r(x_b)$, and $v = w \odot m$, its transpose action is

$$D_f^\top m = \frac{r_0^{-1} + r_1^{-1}}{2} v - \frac{2\bar{x} \langle \bar{x}, v \rangle}{d r_0 r_1 (r_0 + r_1)}. \quad (20)$$

The identity follows from $\Delta r = (2 \langle \bar{x}, \Delta x \rangle / d) / (r_0 + r_1)$ and $\Delta(r^{-1}) = -\Delta r / (r_0 r_1)$. The same construction covers fixed learned scale parameters and the query/key normalization used in the memory path.

B IMPLEMENTATION AND STORAGE ACCOUNTING

Attention. Native endpoint queries, keys, values, and log-normalizers determine the probabilities within each FlashAttention tile. The first sweep accumulates the logarithmic-mean normalizers and weighted centers in Equation 5. Subsequent sweeps accumulate query, key, and value coefficients. Endpoint averages reside in linear-size buffers or are computed inside the tiles.

Memory. The implementation processes 64-token chunks using FLA’s native chunk-state adjoints and batched contractions with saved endpoint states. An associative affine scan computes the finite decay contribution. Convolution and surrounding projections use the installed model primitives and their linear transpose actions.

Storage. For sequence length T , head dimension d , and tile or chunk width C , attention vectors and row statistics occupy $O(Td)$ storage per head; pairwise products reside in bounded tiles. Dense attention arithmetic is $O(T^2d)$. Chunked memory occupies $O(Td + TC + (T/C)d^2)$ auxiliary storage per head for token coefficients, within-chunk products, and boundary states. Both storage expressions are linear in T at fixed d and C . Layer replay supplies intermediate activations, with checkpoint storage accounted for separately.

Execution. Qwen3 uses FP16; Qwen3.5 uses BF16 attention and FLA, with FP32 accumulation where supplied by the underlying operations. Complete attribution cost includes the two endpoint executions, layer replay, contribution propagation, and transfers.

C EVALUATION SPECIFICATION

The FlashTrace `table1-data-v1` release fixes the prompt transformation, eligible source positions, response target, gold mapping, and scoring implementation. DELTATRACE’s scalar target includes the released response and EOS. A source score enters the faithfulness function as $s_i = \max(A_i, 0)$; the released function determines deletion order and normalized contribution mass. Raw signed scores remain available for source visualizations. Full task comparisons use all released examples for the chosen task. Each result record identifies its model, method version, sample selection, metric implementation, and measured execution stages.